Lab 4: Kernel Modules

Student: Ilya-Linh Nguyen

Innopolis main: i.nguen@innopolis.university

Group: CBS-01

Task

Part 1

You need to implement a chardev kernel module int_stack.ko. This kernel module must implement a stack data structure with push/pop operations and also support stack size configuration via ioctl:

- Implement a stack data structure:
    1. Allocate memory dynamically
    2. Use synchronization mechanisms in order to make your module ready for multithreading access (i.e. solve a classic readers-writers problem)
- Implement the following file_operations:
    1. open() and release() - initialization and deinitialization of the stack
    2. read() - pop operation
    3. write() - push operation
    4. ioctl() - configure max stack size
- Error codes must be the same as described in stack(3) manual (see Return codes section). Precisely:
    1. popping from the empty stack -> return NULL
    2. pushing into the full stack -> return -ERANGE errno (error codes must be negative according to kernel modules coding style, you can read this here).
    3. ioctl() errors codes are described in the manual as well
- Ensure that every edge case is handled properly (i.e. stack is empty, stack is full, etc.)

Part 2

Implement a small userspace utility kernel_stack that wraps your module functionality to the following user-friendly CLI:

```
$ kernel_stack set-size 2
$ kernel_stack push 1
$ kernel_stack push 2
$ kernel_stack push 3
ERROR: stack is full
$ kernel_stack pop
2
$ kernel_stack pop
1
```

```
$ kernel_stack pop
NULL

---

$ kernel_stack set-size 3
$ kernel_stack push 1
$ kernel_stack push 2
$ kernel_stack push 3
$ kernel_stack unwind
3
2
1

---

$ kernel_stack set-size 0
ERROR: size should be > 0
$ kernel_stack set-size -1
ERROR: size should be > 0
$ kernel_stack set-size 2
$ kernel_stack push 1
$ kernel_stack push 2
$ kernel_stack push 3
ERROR: stack is full
$ echo $?
-34 # -ERANGE errno code
```

Solution

Here you can see the workflow of the lab and files:

- Asciinema URL: https://asciinema.org/a/uqagCFd7E8huWKIB
- GitHub: https://github.com/ilyalinhnguyen/adv-linux

I did not do screenshots in this lab because Asciinema will show the whole workflow of the program.

Part 1: char device kernel module

The module creates `/dev/int_stack` and implements:

- `open()` / `release()` for device access lifecycle
- `write()` for `push(int)`
- `read()` for `pop()`
- `ioctl(INT_STACK_IOCTL_SET_SIZE)` for max stack size configuration

Implementation details:

- Stack storage is dynamically allocated with `kcalloc`

- Synchronization is implemented with `rw_semaphore` to make access safe across concurrent threads
- Edge cases are handled:
  - Pop from empty stack -> `read()` returns `0` bytes (userspace maps it to `NULL`)
  - Push to full stack -> `write()` returns `-ERANGE`
  - Invalid ioctl command -> `-ENOTTY`
  - Invalid size (`<= 0`) -> `-EINVAL`

Behavior of `set-size`:

- Allocates a new stack with requested capacity
- Resets current content (`top = 0`)

Main state is stored in a single device structure:

- `data`: dynamically allocated `int[]` buffer
- `capacity`: maximum number of elements
- `top`: current size (also next write position)
- `rwsem`: `rw_semaphore` used to protect `data/capacity/top`

`open()` / `release()`:

- Just attach the global device to `file->private_data` and bump an `open_count` (not required for correctness, but useful for debugging).

`write()` (push):

- Expects at least `sizeof(int)` bytes from user.
- Copies one integer from userspace (`copy_from_user`).
- Takes **write** lock (`down_write`) because it modifies `top` and the buffer.
- If `top == capacity` -> returns `-ERANGE` (stack is full).
- Otherwise stores value at `data[top]` and increments `top`.

`read()` (pop):

- Expects at least `sizeof(int)` bytes in userspace buffer.
- Takes **write** lock as well (pop modifies `top`).
- If `top == 0` -> returns `0` bytes (userspace prints `NULL`).
- Otherwise decrements `top`, reads `data[top]`, and copies the integer to userspace (`copy_to_user`).

`ioctl()` (set-size):

- Validates ioctl “magic” and command number; unknown commands return `-ENOTTY`.
- Copies new size from userspace and checks `> 0` (else `-EINVAL`).
- Allocates a new buffer with `kcalloc(new_capacity, sizeof(int), GFP_KERNEL)`.
- Swaps buffers under **write** lock, frees old buffer, and resets `top = 0`.

Synchronization note:

- I used `rw_semaphore` (read-write lock primitive). In this implementation, both push and pop take the **write** lock because they modify the stack. This makes the module safe for concurrent access (no data races/corruption).

Part 2: userspace utility

CLI utility supports:

- `kernel_stack set-size N`
- `kernel_stack push X`
- `kernel_stack pop`
- `kernel_stack unwind`

Behavior:

- On empty pop, prints `NULL`
- On full stack, prints `ERROR: stack is full`
- On invalid size, prints `ERROR: size should be > 0`

`kernel_stack` opens `/dev/int_stack` with `O_RDWR` and maps commands to syscalls:

- `set-size N` -> `ioctl(fd, INT_STACK_IOCTL_SET_SIZE, &N)`
- `push X` -> `write(fd, &X, sizeof(X))`
- `pop` -> `read(fd, &X, sizeof(X))`:
  - if `read()` returns `0` => empty stack => prints `NULL`
- `unwind` -> loop calling `read()` until it returns `0`

Exit codes:

- The utility returns **positive** `errno` values (e.g. `34` for `ERANGE`). In shell, `echo $?` prints `0..255`, so `ERANGE` is observed as `34` (not `-34`).

Quick start

From repository root:

```
cd lab4
make
```

This builds:

- `kernel_stack` (userspace utility)
- `int_stack.ko` (kernel module)

Run:

```
cd lab4
sudo insmod int_stack.ko
```

```
ls -l /dev/int_stack
```

Example workflow 1:

```
./kernel_stack set-size 2
./kernel_stack push 1
./kernel_stack push 2
./kernel_stack push 3
# ERROR: stack is full
./kernel_stack pop
# 2
./kernel_stack pop
# 1
./kernel_stack pop
# NULL
```

Example workflow 2:

```
./kernel_stack set-size 3
./kernel_stack push 1
./kernel_stack push 2
./kernel_stack push 3
./kernel_stack unwind
# 3
# 2
# 1
```

Example workflow 3:

```
./kernel_stack set-size 0
# ERROR: size should be > 0
./kernel_stack set-size -1
# ERROR: size should be > 0
```

Cleanup:

```
sudo rmmod int_stack
make clean
```

That's if for the task.